

On the optimization of GWFA algorithm: enabling real-case applications supporting alignment backtracking

Mirko Coggi, Antonio Sgarlata, Guido Walter Di Donato, Marco Domenico Santambrogio

Dipartimento di Elettronica, Informatica e Bioingegneria

Politecnico di Milano, Milan, Italy

{antonio.sgarlata}@mail.polimi.it

{mirko.coggi, guidowalter.didonato, marco.santambrogio}@polimi.it

Abstract—The Human Pangenome Reference Consortium (HPRC) proved that pangenome graphs represent a population’s genetic variability more efficiently and accurately than linear references. Graphs can intrinsically encode variations as alternative paths inside a directed set of sequence nodes connected by edges. Despite their higher complexity, graph-based genome analysis pipelines are gaining significant interest, and the first sequence-to-graph aligners have already shown improvements in semi-global alignment. However, in pangenomics studies, the global alignment of long reads is fundamental for identifying structural variations and haplotype phasing. In this context, the Graph Wavefront Alignment (GWFA) algorithm emerged as the fastest strategy for aligning long reads to genomic graphs. However, the available GWFA implementation does not support alignment backtracking, a crucial feature in real-case studies. In this paper, we propose a new open-source¹ implementation of the GWFA algorithm that computes and reports the complete traceback in the standard GAF format. Our work achieves a 20× speedup in execution time compared to the state-of-the-art tool GraphAligner and competitive memory usage.

Index Terms—Computational genomics, pangenomics, genome graphs, global alignment

I. INTRODUCTION

Next Generation Sequencing (NGS) technologies significantly decreased the digitalization time of genetic material. This development led to a rapid increase in the amount of genomic data, greatly benefiting genomic research and the field of personalized medicine [1]. Furthermore, NGS has facilitated various new types of previously impossible studies. For instance, *pangenomics* is an emerging area in computational genomics that focuses on analyzing the entire collection of genes (and other regulatory sequences) across all strains of a species or clade, including the core genome and all its variations [2]. However, traditional linear reference genomes are limited to capturing a single consensus sequence and fail to exploit the vast genomic data available from multiple individuals. The Human Pangenome Reference Consortium (HPRC) [3] showed that linear reference genomes tend to result in allele bias, where data that does not match the reference allele is underreported. This bias, particularly evident during read mapping in resequencing, leads to systematic misrepresentation of actual sequences [4], often including population-specific variations. Thus, traditional sequence-based reference models proved insufficient to ensure accurate variant identification.

A promising solution consists of representing the relationships among different genomes in a directed graph, encoding variants as alternative paths. The graph-based reference models surpass the constraints of linear genome representations, allowing for a more precise and comprehensive depiction of genetic variants [5]. Nevertheless, the intrinsic complexity in graph-based computational paradigms presents new technical challenges in developing genome analysis pipelines [6]. The main challenge is the *sequence-to-graph alignment* process, which involves calculating the alignment of a query sequence to a path in the reference graph with the highest alignment score (or lowest distance) among all possible paths. Various strategies and algorithms have been developed to address this challenge, such as GraphAligner [7], Astarix [8], VG Giraffe [6], among others, which achieved excellent performance in semi-global alignment of short reads. However, the global alignment of long reads is crucial for detecting structural variants and haplotype phasing [3].

In the case of linear reference, comparing two sequences of length N was firstly solved in $O(N^2)$ time, but the recent WFA algorithm [9], [10] reduced the search space to a few diagonals in the Dynamic Programming (DP) matrix, leading to $O(ND)$ time, where D is the minimum edit distance between the sequences. For graph-based reference, the graph wavefront algorithm (GWFA) [11] has been proposed. It performs better with exact matching queries, similar to WFA’s improvements to sequence-to-sequence matching. While Zhang et al. [11] proved the efficiency of GWFA by comparing it to the state-of-the-art tool GraphAligner, they did not implement the support for the alignment backtracking. Without this feature, the tool can not report the type of indels occurring in the aligned reads, making their implementation unusable in real-case studies.

For these reasons, we propose a new implementation that improves GWFA by supporting the alignment backtracking, i.e., the computation of the effective sequence of edits to be applied to the query read to match the path in the graph. Our implementation showcases similar memory occupancy to the state-of-the-art solution GraphAligner while attaining a speedup greater than 20×. The rest of the paper is organized as follows: Section II describes the related work in the literature, Section III presents our contribution to GWFA, Section IV discusses the experimental evaluation, and Section V concludes the paper with the main achievements and future works.

¹<https://github.com/sgarlata/gwfa-tb.git>

II. RELATED WORKS

This Section briefly presents the main works in the literature about sequence-to-graph alignment. First, we define a sequence graph as follows: given Σ as a constant-sized alphabet for DNA sequences, i.e., $\Sigma = \{A, C, G, T\}$, a sequence graph $G(V, E, \sigma)$ is a directed graph with vertices V and edges E . The function $\sigma : V \rightarrow \Sigma^*$ labels each vertex $v \in V$ with a string $\sigma(v)$ over the alphabet Σ .

Rautiainen et al. [7] proposed the first banded sequence-to-graph alignment algorithm to achieve human-scale genome size analysis. Its implementation is called GraphAligner and uses a Seed-and-Extend method with a minimizer-based seeding, clustering seeds with superbubbles, and finally aligning with a bit-parallel banded DP algorithm leveraging inter-sequence parallelism. The authors employed bit-vectors to increase performance; however, this reduces flexibility, as GraphAligner can only compute the edit distance between the query sequence and the graph with a fixed scoring matrix. The resulting time for their proposed algorithm is $O(|V| + \lceil \frac{m}{w} \rceil |E| \log w)$ for acyclic graphs, and $O(|V| + m|E| \log w)$ for cyclic graphs, with w as the *bit-word* length, usually 64-bit.

Another solution is the AStarix [12] aligner, based on a Seed-and-Extend version of the A* algorithm, a generalized variant of Dijkstra's shortest-path search. AStarix prioritizes exploring nodes that seem closer to the target nodes, improving the performance of Dijkstra's algorithm while remaining optimal. To optimize the seeding process, Astarix splits the query read in different substrings and then maps them on the graph, highlighting the most promising paths and, thus, reducing the compute space. This tool is not multi-threaded and does not involve banded heuristics, but it allows choosing the edits' scores with a customizable scoring matrix. The algorithm has a $O(V + mE)$ time complexity, common in shortest-path search. Despite the higher flexibility of the tool, AStarix fails to index highly branched graphs, making it unscalable in population studies. This represents a massive limitation because, as we can sequence and collect more and more genomic data, pangenome graphs are naturally getting bigger and more branched. Thus, even if it is an interesting solution for some small and linear graphs, Astarix will not be able to face future challenges.

More recently, Zhang et al. [11] proposed the Graph Wavefront Alignment (GWFA) algorithm, adapting the WFA [9] algorithm for sequence-to-sequence alignment to graph references. GWFA aligns sequences globally, and its speed increases when the read perfectly matches the graph, leading to $O(m)$ time in this best-case scenario. GWFA creates a $m \times v$ DP matrix for each graph node (where v is the length of a sequence node). Nodes are topologically ordered, and the algorithm fills the DP matrices starting from the first node and propagating to other nodes when the right border of a node's DP matrix is reached. GWFA tries to iteratively compute the DP cells with the same distance score, e.g., in the first iteration, all the 0-scored cells are computed; in the second iteration, all the 1-scored ones, and so on. The alignment process

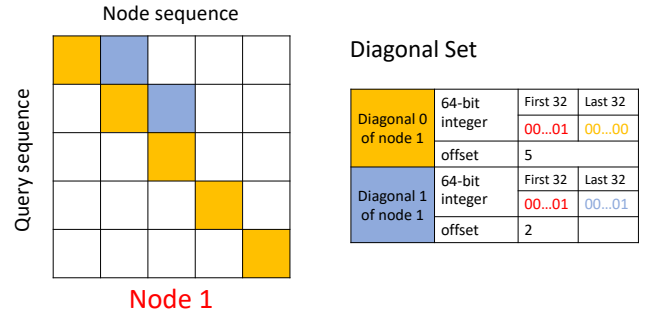


Fig. 1. The diagonals are identified by a 64-bit integer where the first 32 bits stand for the graph node, and the last 32 bits are for identifying the diagonal inside the node. The offset represents the length of the computed diagonal at a specific iteration.

concludes when a cell in the final row of a node's DP matrix is calculated, so when the last character of the query is reached. More details on the algorithm's execution are provided in section III. The authors also proposed a pruning heuristic that drops the worst diagonals, achieving additional speedup on large graphs. The implemented version of this algorithm does not compute the traceback, so it does not provide any information regarding how the query differs from the reference graph². Zhang et al. [11] showed how GWFA implementation outperforms the state-of-the-art tool GraphAligner in execution time and memory footprint. However, the comparison was not completely fair because GraphAligner includes the non-negligible computation of the traceback in the execution time, while this step is missing in GWFA.

III. METHODOLOGY AND IMPLEMENTATION

This section briefly describes the GWFA algorithm and our contribution to creating a new implementation usable in real application scenarios.

A. Graph Wavefront Alignment

As introduced in the previous section, the algorithm works with the diagonals of multiple DP matrices (one matrix for each graph node). The algorithm starts by comparing the query sequence with the first node of the reference graph. The edit distance is initially set to 0 and increases by one at each iteration. Calculating the cells along a diagonal is called *extension* and identifies matches and substitutions. Every time a mismatch is found, the *extension* procedure stops, and the *expansion* phase begins. It adds the two adjacent diagonals to the computation, introducing the insertion and deletion variations. This step increases the number of diagonals at each iteration and, consequently, the size of the wavefront, i.e., the collection of all the last cells computed in each diagonal.

Diagonals are stored in two structures: a set and a queue. The queue accounts for all the diagonals computed during the current cycle. The set instead stores all the diagonals that will be computed in the next iteration. For this reason, before

²<https://github.com/lh3/gwfa>

starting each iteration, the set is copied inside the queue, and the diagonals are popped out of the queue and processed, i.e., extended and eventually expanded. The cycle ends when the queue is empty.

When a diagonal reaches the right border of the DP matrix, the algorithm aligns the remaining query characters by investigating the neighboring nodes in the graph. First, the algorithm checks each adjacent node to identify where a match can be extended. For such nodes, the corresponding diagonals are pushed to the end of the queue. Then, all other non-matching neighboring nodes are processed similarly, but the new diagonals are pushed at the end of the set to be processed at the next iteration. The alignment is completed when the wavefront reaches the end of the query sequence along one of the DP matrices' diagonals, and the edit distance is reported.

B. GWFA-TraceBack Implementation

We kept the diagonal structure from Zhang et al.'s implementation, comprising a 64-bit integer and an offset k . The first 32 bits of the integer are for the graph node identifier, and the last 32 are for the diagonal identifier inside the node. The offset k instead stands for the length of the diagonal, so the distance of the last element from the beginning of the diagonal (Figure 1).

Our solution improves the previous implementation by exploiting a set of new structures to store the information related to how the query maps on the graph, saving the sequence of operations in a CIGAR string. In particular, we implemented a hash table that associates each *path* in the graph to the corresponding CIGAR string, updated at each iteration until the end of the alignment. Specifically, a *path* is seen as a collection of diagonals from subsequent nodes' matrices, but different paths can share the same diagonal, for example, in the case of graph bifurcations. Carrying on this information during the process needed ad hoc structures to keep the memory occupancy low. For this reason, we implemented the CIGAR string as a vector of 16-bit integers supporting bit-packing operations. Specifically, the first two bits of the integer are used to encode the operation: 00 for match, 01 for mismatch, 10 for deletion, and 11 for insertion. The remaining 14 bits, instead, encode the length of that operation. An example is provided in Figure 2.

Updating the CIGAR strings works as follows. During the *extension* step, our implementation pushes a matching element to the end of the corresponding CIGAR vector and updates its length until the alignment matches. In the case of *expansion*, new diagonals are added to the wavefront. Thus, new graph paths are investigated, and consequently, new CIGAR vectors are created starting from the CIGAR string of the diagonal from which the *expansion* is generated. Once the alignment is completed, the CIGAR string of the optimal alignment is provided by the path that reaches the lower border of the matrix.

To compare our findings with those of other cutting-edge solutions, we condensed the alignment data into a GAF file, the most commonly used format for alignment output [4]. This

A)			B)		
			CIGAR: 30=1D20=1I3X10=		
Operation	CIGAR character	Bit-packing	CIGAR	16-bit integer	
				First 2 bits	Last 14 bits
Match	=	00	30=	00	00..11110
Mismatch	X	01	1D	10	00..00001
Deletion	D	10	20=	00	00..10100
Insertion	I	11	1I	11	00..00001
			3X	01	00..00011
			10=	00	00..01010

Fig. 2. A) Recap of the bit coding of the different edits operations that can occur during the alignment. B) An example of encoding a CIGAR string (30=1D20=1I3X10=) with the described bit-packing technique. The first two bits encode the operation, while the last 14 bits encode the length of that operation.

approach makes our implementation well-suited for real case studies and allows for a fair comparison with GraphAligner.

IV. EXPERIMENTAL EVALUATION

We compared our implementation (GWFA-TB), which supports backtracking, against the one proposed by Zhang et al. [11] (GWFA), and the state-of-the-art tool GraphAligner [7]. As already explained in the GWFA work [11], Astarix reported errors or could not finish any of the experiments of this study. For these reasons, we excluded it from the comparison. The dataset used in the experiments comprised a graph reference and haplotype reads of the *Major Histocompatibility Complex* (MHC), published by GWFA's authors³. In particular, the graph presents 980 sequence nodes for a total of over 5 Mb and 1399 edges. The reads set comprises 60 haplotype sequences of 5 Mb mean length with a mean edit distance of 29524 (computed with Zhang et al. implementation). All experiments were run on an AMD EPYC 7763 64-core CPU (2.4 GHz) machine equipped with 128 GB main memory.

To conduct a fair evaluation, we wanted to test all the tools with optimal and heuristic modes. Unfortunately, GraphAligner only implements a banded alignment, which drops the paths that exceed a customizable threshold. Thus, given that the maximum edit distance was 34598, we set all the heuristics bandwidth to 35000 to guarantee the optimal alignment of all the reads in the dataset. Furthermore, we tested the tools using the two bandwidth values employed in GWFA's original paper (20000 and 30000). Figures 3 and 4 display the results of single-threaded executions of the three solutions (GWFA, GWFA-TB, GraphAligner) on the MHC data. In particular, Figure 3 shows the total execution time. GWFA is faster than our solution because it only reports the alignment score and does not compute the alignment backtrace. Compared to GraphAligner, our implementation achieves a 23× speedup with a bandwidth value of 20000, a 22× speedup with a band of 30000, and a 20× speedup

³<https://zenodo.org/records/6056061>

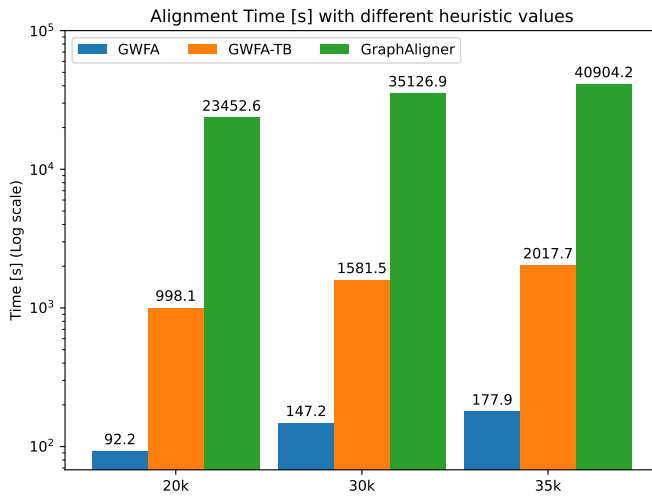


Fig. 3. Execution time of the alignment of 60 MHC haplotypes to the reference graph. The three tools have been run with different heuristic values. The time is reported in seconds, and the scale is logarithmic.

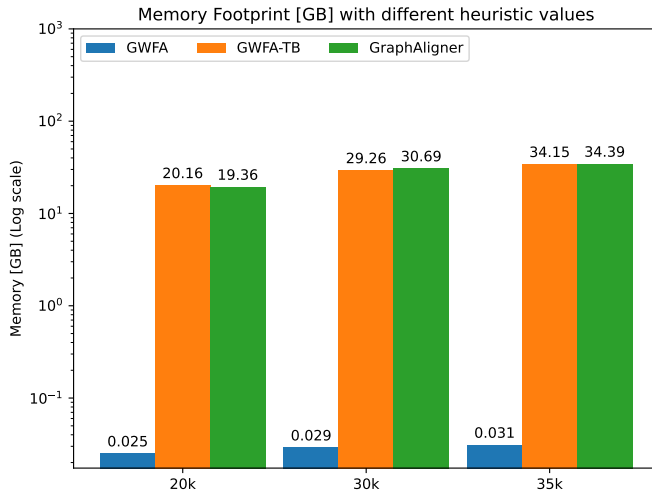


Fig. 4. Memory occupancy of the alignment of 60 MHC haplotypes to the reference graph. The three tools have been run with different heuristic values. The occupancy is reported in gigabytes, and the scale is logarithmic.

with a band of 35000. On the other side, Figure 4 shows the memory occupancy of the three tools. Because of the additional CIGAR computation, our solution has a higher memory footprint than the original GWFA implementation, but it is very similar to GraphAligner’s occupancy with all the heuristic values. To verify the correctness of the alignment, we compared the scores computed by our solution and the original implementation, which was already validated by Zhang et al. [11]. Since our contributions do not change the actual computation of scores, GWFA-TB reported the same values computed by GWFA. This proves that our tool provides better runtime performances than the state-of-the-art, without compromises in accuracy and memory occupancy.

V. CONCLUSIONS

In this paper, we tackle the challenge of aligning long genome sequences to pangenome graphs to improve the detection of structural variations and haplotype phasing. In particular, we proposed a new implementation of the GWFA algorithm that supports alignment backtracking, reporting all the mapping information and making it usable in real-case studies. Specifically, we implemented a tool for computing the CIGAR string and reporting all the alignment information in the standard GAF format. Our solution shows a comparable memory footprint and more than $20\times$ speedup in the execution time of global alignments on MHC data compared to the state-of-the-art tool GraphAligner. Further developments in this work will regard the creation of lighter structures to reduce memory occupancy and improve the execution time by investigating the algorithm parallelisms and leveraging hardware acceleration.

ACKNOWLEDGMENT

This work was partly supported by Oracle Cloud credits and resources provided by the Oracle for Research program.

REFERENCES

- [1] I. R. König, O. Fuchs, G. Hansen, E. von Mutius, and M. V. Kopp, “What is precision medicine?” *European respiratory journal*, vol. 50, no. 4, p. 1700391, 2017.
- [2] B. E. Dutilh and C. P. G. Consortium, “Computational pan-genomics: status, promises and challenges,” *Briefings in bioinformatics*, vol. 19, no. 1, pp. 118–135, 2018.
- [3] T. Wang, L. Antonacci-Fulton, K. Howe, H. A. Lawson, J. K. Lucas, A. M. Phillippy, A. B. Popejoy, M. Asri, C. Carson, M. J. Chaisson et al., “The human pangenome project: a global resource to map genomic diversity,” *Nature*, vol. 604, no. 7906, pp. 437–446, 2022.
- [4] V. Singh, S. Pandey, and A. Bhardwaj, “From the reference human genome to human pangenome: Premise, promise and challenge,” *Frontiers in Genetics*, vol. 13, p. 1042550, 2022.
- [5] J. A. Baaijens, P. Bonizzoni, C. Boucher, G. Della Vedova, Y. Pirola, R. Rizzi, and J. Sirén, “Computational graph pangenomics: a tutorial on data structures and their applications,” *Natural computing*, vol. 21, no. 1, pp. 81–108, 2022.
- [6] E. Garrison, J. Sirén, A. M. Novak, G. Hickey, J. M. Eizenga, E. T. Dawson, W. Jones, S. Garg, C. Markello, M. F. Lin et al., “Variation graph toolkit improves read mapping by representing genetic variation in the reference,” *Nature biotechnology*, vol. 36, no. 9, pp. 875–879, 2018.
- [7] M. Rautiainen and T. Marschall, “Graphaligner: rapid and versatile sequence-to-graph alignment,” *Genome biology*, vol. 21, no. 1, p. 253, 2020.
- [8] P. Ivanov, B. Bichsel, H. Mustafa, A. Kahles, G. Rätsch, and M. Vechev, “Astarix: Fast and optimal sequence-to-graph alignment,” in *Research in Computational Molecular Biology*, R. Schwartz, Ed. Cham: Springer International Publishing, 2020, pp. 104–119.
- [9] B. Branchini, G. Gerometta, L. Cicolini, A. Zeni, E. Del Sozzo, and M. D. Santambrogio, “Surfing the wavefront of genome alignment,” in *2022 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2022, pp. 1754–1758.
- [10] S. Marco-Sola, J. C. Moure, M. Moreto, and A. Espinosa, “Fast gap-affine pairwise alignment using the wavefront algorithm,” *Bioinformatics*, vol. 37, no. 4, pp. 456–463, 2021.
- [11] H. Zhang, S. Wu, S. Aluru, and H. Li, “Fast sequence to graph alignment using the graph wavefront algorithm,” *arXiv preprint arXiv:2206.13574*, 2022.
- [12] P. Ivanov, B. Bichsel, and M. Vechev, “Fast and optimal sequence-to-graph alignment guided by seeds,” in *International Conference on Research in Computational Molecular Biology*. Springer, 2022, pp. 306–325.